

CSE563 — Concept Sheet, Block D

Unit V: Algorithm Complexity Analysis

Unit VI: Algorithmic and Optimization Approaches

Theory reference — every concept, no numericals

School of Computer Science & Engineering

Session 2025–26



What This Sheet Is — and Why These Two Units Are Taught Together

A **theory-only** reference for Units V and VI, delivered as one teaching block because they are two halves of a single subject: Unit V tells you *what an algorithm costs*, Unit VI tells you *how to design one*. The dependencies run both ways:

- Recurrences and the master theorem (V) are the tool for analysing divide-and-conquer (VI).
- NP-completeness (V) is the *reason* metaheuristics (VI) exist at all.
- Optimal substructure appears in V's analysis and in VI's design of dynamic programming.

Calculations live in the *Numerical Guides*; practice questions live in the *Worksheets*.

Contents

Unit V — Algorithm Complexity Analysis

1 Measuring Running Time

The Model and the Cases

Cost model — count *elementary operations* — comparison, assignment, arithmetic, array access, each $O(1)$ on the RAM model — as a function $T(n)$ of the input size.

Why we discard constants — they depend on the compiler, the processor and the memory hierarchy. What survives is the *growth rate*, which is a property of the algorithm itself.

Worst case — the maximum over all inputs of size n . This is the default, because it is a *guarantee*.

Average case — requires assuming a probability distribution over inputs, which is often unrealistic.

Best case — rarely useful on its own — it describes a lucky input, not the algorithm.

Space complexity — extra memory beyond the input. An algorithm using $O(1)$ extra space is *in place*.

Amortised analysis — the average cost per operation over a *sequence*, used when occasional expensive operations are offset by many cheap ones — e.g. dynamic array resizing, which is $O(1)$ amortised despite occasional $O(n)$ copies.

2 Asymptotic Notation

The Five Notations

Notation	Definition	Read as
$f = O(g)$	$\exists c, n_0 : f(n) \leq cg(n)$ for $n \geq n_0$	grows no faster than
$f = \Omega(g)$	$\exists c, n_0 : f(n) \geq cg(n)$ for $n \geq n_0$	grows at least as fast as
$f = \Theta(g)$	both of the above	same order as
$f = o(g)$	$\lim f/g = 0$	grows strictly slower than
$f = \omega(g)$	$\lim f/g = \infty$	grows strictly faster than

The limit test — if $\lim_{n \rightarrow \infty} f/g = c$ with $0 < c < \infty$ then $f = \Theta(g)$; if 0 then $f = o(g)$; if ∞ then $f = \omega(g)$. L'Hôpital's rule (Unit I) evaluates the limit.

The Growth Hierarchy

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

Class	Name	Typical algorithm
$O(1)$	constant	hash lookup, array index
$O(\log n)$	logarithmic	binary search, balanced BST
$O(n)$	linear	one pass over the data
$O(n \log n)$	linearithmic	merge sort, heap sort, FFT
$O(n^2)$	quadratic	insertion sort, all-pairs check
$O(n^3)$	cubic	naive matrix multiply, Floyd–Warshall
$O(2^n)$	exponential	brute-force subsets
$O(n!)$	factorial	brute-force permutations (TSP)

Scale intuition at 10^9 operations per second and $n = 10^6$: $n \log n \approx 0.02$ s; $n^2 \approx 17$ minutes; $n^3 \approx 31,700$ years.

Rules and Misconceptions

- **Drop constants and lower-order terms:** $5n^3 + 100n^2 + 7 = \Theta(n^3)$.
- **Sum rule** (sequential code): $O(f) + O(g) = O(\max(f, g))$. **Product rule** (nested): $O(f) \cdot O(g) = O(fg)$.
- **Logarithm base is irrelevant:** $\log_a n = \log_b n / \log_b a$, a constant factor. But the base of an *exponential* matters: 3^n is not $\Theta(2^n)$.
- **Polynomial beats polylogarithmic:** $\log^k n = o(n^\varepsilon)$ for every k and every $\varepsilon > 0$. **Exponential beats polynomial:** $n^k = o(2^n)$ for every k .

Key misconception — O does *not* mean “worst case” and Ω does not mean “best case”. They are bounds on *any* function you choose to analyse. Insertion sort is $\Theta(n^2)$ in the worst case and $\Theta(n)$ in the best case; saying it is “ $O(n^2)$ ” is a true but loose statement about the algorithm overall.

Second misconception — $O(n^2)$ is technically true of a linear algorithm, since O is only an upper bound. Use Θ when a tight bound is known.

3 Recurrence Relations

Forming and Classifying

Recurrence — defines $T(n)$ in terms of T at smaller arguments, plus base cases. Read it off the code as (number of recursive calls) $\times T$ (subproblem size) + (non-recursive work).

The decisive classification — is the subproblem n/b (**divisive** — the master theorem may apply) or $n - k$ (**subtractive** — iterate instead)? Deciding this first prevents most wasted effort.

Algorithm	Recurrence	Solution
Binary search	$T(n/2) + 1$	$\Theta(\log n)$
Merge sort	$2T(n/2) + n$	$\Theta(n \log n)$
Tower of Hanoi	$2T(n - 1) + 1$	$\Theta(2^n)$
Selection sort (recursive)	$T(n - 1) + n$	$\Theta(n^2)$
Karatsuba	$3T(n/2) + n$	$\Theta(n^{1.585})$
Strassen	$7T(n/2) + n^2$	$\Theta(n^{2.807})$
Naive Fibonacci	$T(n - 1) + T(n - 2) + 1$	$\Theta(\varphi^n)$

The Three Solution Methods

Iteration (unrolling) — substitute the recurrence into itself until a pattern emerges, express the general term after k steps, set k so the argument reaches the base case, and sum. Best for subtractive recurrences.

Recursion tree — draw the calls; level i has a^i nodes each costing $f(n/b^i)$. Sum per level, then across the $\log_b n$ levels. Excellent for *intuition* and for recurrences the master theorem cannot handle.

Substitution — guess the answer and prove it by induction (Unit I). This is the only method that constitutes a **proof** — the other two produce a well-founded guess.

Linear Recurrences with Constant Coefficients

For $a_n = c_1 a_{n-1} + c_2 a_{n-2}$, substitute $a_n = r^n$ to obtain the **characteristic equation** $r^2 - c_1 r - c_2 = 0$.

- **Distinct roots** $r_1 \neq r_2$: $a_n = \alpha r_1^n + \beta r_2^n$.
- **Repeated root** r : $a_n = (\alpha + \beta n) r^n$.

Fit α, β to the initial conditions, then verify on a known term. The *larger* root governs the

growth rate.

Fibonacci: $r^2 - r - 1 = 0$ gives $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$, so $F_n = \Theta(\varphi^n)$ — which is exactly why naive recursive Fibonacci is exponential and why memoisation is transformative.

The Master Theorem

For $T(n) = aT(n/b) + f(n)$ with constants $a \geq 1$, $b > 1$, let $d = \log_b a$ (so n^d is the total cost of the leaves). Compare $f(n)$ with n^d :

Case 1 $f(n) = O(n^{d-\varepsilon})$ for some $\varepsilon > 0$ — the *leaves* dominate: $T(n) = \Theta(n^d)$.

Case 2 $f(n) = \Theta(n^d)$ — every level costs the same: $T(n) = \Theta(n^d \log n)$.

Case 3 $f(n) = \Omega(n^{d+\varepsilon})$ and the regularity condition $af(n/b) \leq kf(n)$ holds for some $k < 1$ — the *root* dominates: $T(n) = \Theta(f(n))$.

The intuition — the tree has $\log_b n$ levels; level i holds a^i nodes each costing $f(n/b^i)$; the leaves number $a^{\log_b n} = n^{\log_b a}$. The theorem merely asks which level wins.

When it fails — (i) the gap in Case 1 or 3 is only logarithmic, not polynomial — e.g. $2T(n/2) + n \log n$; (ii) the recurrence is subtractive; (iii) a or b is not constant; (iv) the subproblems have unequal sizes.

Extended form — if $f(n) = \Theta(n^d \log^k n)$ with $d = \log_b a$ and $k \geq 0$, then $T(n) = \Theta(n^d \log^{k+1} n)$ — this covers the common gap case.

4 Complexity Classes

P, NP, NP-hard, NP-complete

All defined for **decision problems** (yes/no answers). An optimisation problem is converted by adding a threshold: “find the shortest tour” becomes “is there a tour of length $\leq k$?”

P — solvable by a deterministic algorithm in polynomial time $O(n^k)$. Examples: sorting, shortest paths, max flow, MST, 2-SAT, primality (AKS, 2002), linear programming.

NP — a proposed *yes*-answer (a **certificate**) can be *verified* in polynomial time. Equivalently, solvable in polynomial time by a machine that may guess.

NP-hard — at least as hard as everything in NP — every NP problem reduces to it in polynomial time. It need not be in NP (the halting problem is NP-hard but undecidable).

NP-complete — in NP *and* NP-hard — the hardest problems *within* NP. If any single one is in P, then $P = NP$.

Relationship: $P \subseteq NP$. Whether $P = NP$ is open, a Clay Millennium Prize problem, and almost universally believed false.

Reductions and How to Prove NP-Completeness

Polynomial reduction $A \leq_p B$ — a polynomial-time function mapping instances of A to instances of B preserving the yes/no answer. It means “ B is at least as hard as A ”. If $B \in P$ then $A \in P$.

To prove X is NP-complete:

1. Show $X \in NP$ — give a certificate and a polynomial verifier.
2. Pick a *known* NP-complete Y and show $Y \leq_p X$ — reduce the known hard problem **to** the new one.

Direction matters — reducing $X \leq_p Y$ would only show X is no harder than Y , which proves nothing about hardness. Reversing the direction is the commonest error in these proofs.

Cook–Levin theorem (1971) — **SAT** is NP-complete — the first such proof, made directly from the definition. Karp (1972) then reduced SAT to 21 further problems, and every later proof

chains back to that root.

The Canonical NP-Complete Problems — and Their Easy Cousins

NP-complete	The question	Easy cousin (in P)
SAT / 3-SAT	is there a satisfying assignment?	2-SAT, Horn-SAT
Vertex cover	a set of $\leq k$ vertices touching every edge?	on bipartite graphs (König)
Clique / Independent set	a complete (or edgeless) set of size $\geq k$?	—
Hamiltonian cycle / TSP	a cycle through all vertices?	Eulerian circuit
Graph 3-colouring	is $\chi(G) \leq 3$?	2-colouring (bipartite test)
Subset sum / Knapsack	a subset summing to exactly k ?	pseudo-polynomial DP

The lesson of the right-hand column: a very small change in the problem statement can cross the tractability boundary. Euler is a local degree check; Hamilton is global. 2-colouring is a BFS parity test; 3-colouring is NP-complete.

Satisfiability

CNF — a conjunction (AND) of **clauses**, each a disjunction (OR) of **literals**. k -SAT restricts every clause to k literals.

Where the boundary lies — **2-SAT** is in P — build the implication graph (clause $(a \vee b)$ gives $\neg a \rightarrow b$ and $\neg b \rightarrow a$) and check no variable shares a strongly connected component with its negation, in $O(n + m)$. **3-SAT** is NP-complete and is the usual starting point for reductions.

Horn-SAT (at most one positive literal per clause — the logic-programming fragment) is in P by unit propagation.

DPLL — the practical method: **unit propagation** (a clause with one unassigned literal forces it), **pure literal elimination**, then **branch** and backtrack on conflict. Modern CDCL solvers add clause learning and non-chronological backjumping. This is backtracking (Unit VI) plus the resolution rule (Unit I).

Practice versus theory — worst-case hardness does not prevent success on structured industrial instances with millions of variables — hardware verification, dependency resolution, planning and timetabling are all solved by SAT solvers daily.

5 Searching and Sorting

Searching

Linear search — $\Theta(n)$; works on unsorted data.

Binary search — $\Theta(\log n)$, requiring *sorted* data. At most $\lfloor \log_2 n \rfloor + 1$ comparisons. Recurrence $T(n) = T(n/2) + 1$.

Lower bound — any comparison-based search needs $\Omega(\log n)$ comparisons — a decision tree with n leaves has height at least $\log_2 n$. Binary search therefore is optimal.

Sorting — the Comparison Table

Algorithm	Best	Avg	Worst	Space	Stable	Notes
Bubble	n	n^2	n^2	1	yes	teaching only
Selection	n^2	n^2	n^2	1	no	fewest swaps ($n - 1$)
Insertion	n	n^2	n^2	1	yes	best on small or nearly-sorted input; online
Merge	$n \log n$	$n \log n$	$n \log n$	n	yes	guaranteed; external sorting; linked lists
Quick	$n \log n$	$n \log n$	n^2	$\log n$	no	fastest in practice; worst case on sorted input with a bad pivot
Heap	$n \log n$	$n \log n$	$n \log n$	1	no	in-place guarantee; priority queues
Counting	$n + k$	$n + k$	$n + k$	$n + k$	yes	integers in $[0, k]$; not comparison-based
Radix		$d(n + k)$		$n + k$	yes	d digits; counting sort per digit

Stability — equal keys retain their original relative order — essential when sorting by one key after another.

Library implementations — Python’s `sorted` is Timsort (merge + insertion, stable); C++ `std::sort` is introsort (quick + heap + insertion), which switches to heap sort once the recursion depth exceeds $\sim 2 \log n$ to guarantee $O(n \log n)$.

The $\Omega(n \log n)$ Lower Bound

Model any comparison sort as a binary **decision tree**: each internal node is one comparison with two outcomes, each leaf one of the $n!$ possible output orderings. A binary tree with $n!$ leaves has height

$$h \geq \log_2(n!) \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n),$$

and h is the worst-case number of comparisons. Hence no comparison sort can do better, and merge sort and heap sort are asymptotically **optimal**.

Why counting and radix sort escape it. They never compare two keys — they use the key as an *array index*. This requires the keys to be integers in a bounded range, so the decision-tree model simply does not apply. The bound constrains a *model*, not sorting as such.

Unit VI — Algorithmic and Optimization Approaches

6 Choosing a Paradigm

The Five Paradigms at a Glance

Paradigm	Idea	Optimal?	Use when
Greedy	best local choice, never revisited	only if the greedy-choice property holds	provably safe choices exist (MST, Huffman, activity selection)
Divide & conquer	split, solve independently, combine	yes	subproblems are <i>disjoint</i>
Dynamic programming	solve overlapping subproblems once	yes	optimal substructure + <i>overlapping</i> subproblems
Backtracking	DFS over partial solutions, prune	yes (exhaustive)	constraint satisfaction; small NP-hard instances
Metaheuristics	population or trajectory evolved stochastically	no	large NP-hard or black-box problems

7 Greedy Approach

The Two Required Properties

Greedy-choice property — a locally optimal choice is contained in *some* globally optimal solution, so committing to it never rules out optimality. Proved by an **exchange argument**: take any optimal solution, swap in the greedy choice, and show the result is no worse.

Optimal substructure — an optimal solution to the whole contains optimal solutions to its subproblems.

If you cannot prove the greedy-choice property, do not use greedy — use dynamic programming.

Where Greedy Works, and Where It Fails

Problem	Greedy rule	Status
Activity selection	earliest <i>finish</i> time	optimal
Fractional knapsack	highest value/weight ratio	optimal
Huffman coding	merge the two least-frequent nodes	optimal among prefix codes
MST (Kruskal, Prim)	lightest safe edge (cut property)	optimal
Dijkstra	closest unvisited vertex	optimal for non-negative weights
Job scheduling for mean completion	shortest job first	optimal
0/1 knapsack	highest ratio	fails — items cannot be split to fill leftover capacity
Coin change, non-canonical system	largest coin first	fails — e.g. $\{1, 3, 4\}$ for 6 gives 3 coins, optimum is 2
Graph colouring (Welsh–Powell)	highest degree first	heuristic only — need not reach $\chi(G)$

Huffman coding — builds an optimal prefix code by repeatedly merging the two least-frequent nodes; $O(n \log n)$ with a min-heap. It is the entropy-coding stage of JPEG, MP3 and ZIP. The *prefix property* — no codeword is a prefix of another — is what makes the bit stream uniquely decodable without separators.

8 Divide and Conquer

Template and Analysis

1. **Divide** into a subproblems of size n/b .
2. **Conquer** each recursively (base case solved directly).
3. **Combine** the sub-solutions in $f(n)$ time.

This yields $T(n) = aT(n/b) + f(n)$, which the master theorem (Unit V) solves directly.

The essential requirement — the subproblems must be **independent**. If they overlap, plain recursion re-solves them exponentially often and dynamic programming is needed instead.

How speedups are achieved — by reducing the branching factor a , since Case 1 gives $\Theta(n^{\log_b a})$. Karatsuba replaces 4 half-size products with 3, turning $\Theta(n^2)$ into $\Theta(n^{1.585})$; Strassen replaces 8 with 7, turning $\Theta(n^3)$ into $\Theta(n^{2.807})$.

Standard examples — binary search, merge sort, quicksort, maximum subarray, closest pair of points, FFT, Karatsuba, Strassen.

9 Dynamic Programming

When to Use It, and the Five Design Steps

Two required properties: **optimal substructure** (an optimal solution is built from optimal sub-solutions) *and* **overlapping subproblems** (the same subproblem recurs, so naive recursion is exponentially wasteful).

The five steps.

1. **Define the state** — what does $dp[i]$ or $dp[i][j]$ mean?
2. **Write the recurrence** relating a state to smaller states.
3. **Set the base cases.**
4. **Choose an evaluation order** — memoisation or tabulation.
5. **Reconstruct the solution** by tracing which choice produced each value.

Time = (number of states) $\times$ (work per state).

Memoisation versus Tabulation

	Memoisation (top-down)	Tabulation (bottom-up)
Form	natural recursion + a cache	iterative loops from the base cases
Subproblems solved	only those <i>reachable</i>	all of them
Overhead	recursion stack, call cost	none
Risk	stack overflow at depth	wasted states
Space optimisation	harder	easy — often only the previous row is needed

Both give the same asymptotic time.

The Standard DP Problems

Problem	State	Recurrence
0/1 knapsack	$dp[i][c]$: best value, first i items, capacity c	$\max(dp[i-1][c], dp[i-1][c-w_i] + v_i)$
LCS	$L[i][j]$: LCS of the two prefixes	$\text{match} \Rightarrow L[i-1][j-1] + 1$; else $\max(L[i-1][j], L[i][j-1])$
Coin change	$dp[a]$: fewest coins for a	$1 + \min_{c \leq a} dp[a-c]$
Floyd–Warshall	$d^{(k)}[i][j]$	$\min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$

Others: edit distance, longest increasing subsequence, matrix-chain multiplication, rod cutting, Bellman–Ford, Viterbi, CKY parsing.

Pseudo-polynomial complexity — knapsack’s $\Theta(nW)$ is polynomial in the *value* W but exponential in its *bit-length* $\log_2 W$. This is why 0/1 knapsack remains NP-complete despite the table algorithm, and why the DP is practical only for modest W .

DP versus greedy — DP evaluates all choices at each step and keeps the best; greedy commits to one and never reconsiders.

10 Backtracking

Mechanism and Scope

Definition — a depth-first search over the tree of *partial* solutions. Extend by one decision; as soon as a constraint is violated, **prune** the branch and undo the last decision.

Why it is practical — the worst case is exponential — it explores the whole tree — but a cheap feasibility test can eliminate an entire subtree at once. For 8-Queens, brute force examines $8^8 \approx 1.7 \times 10^7$ placements while backtracking explores on the order of 2×10^3 nodes.

Branch and bound — the optimisation variant. It additionally computes a *bound* on the best value attainable in a branch and prunes any branch that cannot beat the incumbent. Backtracking prunes on **infeasibility**; branch and bound also prunes on **sub-optimality**.

Applications — N -Queens, Sudoku, graph colouring, Hamiltonian cycle, subset sum, DPLL SAT solving, regular-expression engines, constraint programming, and game-tree search (minimax with alpha–beta pruning is backtracking with bounds).

11 Genetic Algorithms

Components of a GA

A **genetic algorithm** maintains a *population* of candidate solutions, scores them by a *fitness function*, and produces the next generation by selection, crossover and mutation. It requires no gradient, no convexity and no continuity — only the ability to evaluate fitness.

1. **Representation** — binary string, integer vector, permutation (for TSP), real vector, or tree (genetic programming). *The encoding must suit the operators.*
2. **Initial population** — typically 50–500 random individuals.
3. **Fitness function** — the objective to maximise.
4. **Selection** — roulette wheel ($p_i = f_i / \sum f$), tournament (pick k , keep the best), or rank-based. **Elitism** copies the best individuals through unchanged.
5. **Crossover** (rate ≈ 0.6 – 0.9) — single-point, two-point, uniform; order crossover or PMX for permutations.
6. **Mutation** (rate ≈ 0.001 – 0.05 per gene) — bit flip, perturbation, or swap.
7. **Termination** — a generation limit, a fitness threshold, or stagnation.

Failure Modes and Their Remedies

Premature convergence — one very fit individual takes over the mating pool; crossover between near-identical parents produces nothing new and the search stalls at a local optimum. *Remedies:* tournament or rank selection (which depend on order, not magnitude), fitness scaling, higher mutation, a larger population, or niching.

Bad encoding — single-point crossover of two TSP tours represented as plain lists produces invalid tours with repeated cities. *Remedy:* permutation-preserving operators.

Fitness scaling sensitivity — with roulette selection, adding a constant to every fitness changes the selection pressure — a sign that the raw values, not just the ranking, are driving the search.

No guarantee — a GA can return a poor solution and gives no indication that it has. *Remedy:* run several random seeds and compare against a bound.

The role of mutation — it is as likely to harm as to help on any single application. Its purpose is maintaining *diversity*, not immediate improvement — which is why rates are low and elitism protects the best individual.

12 Nature-Inspired Algorithms

The Common Principle — Exploration versus Exploitation

Every metaheuristic balances **exploration** (sampling new regions, so as not to be trapped in a local optimum) against **exploitation** (refining the best solutions found so far). The metaphors differ; the underlying trade-off does not.

No Free Lunch theorem — (Wolpert & Macready, 1997) averaged over *all* possible objective functions, every optimiser performs identically. Superiority comes only from matching the algorithm to the *structure* of the problem at hand — there is no universally best method.

The Main Families

Algorithm	Inspiration	Key mechanism	Best suited to
Genetic algorithm	Darwinian evolution	selection, crossover, mutation on a population	discrete, permutations, black-box
Simulated annealing	metallurgy	single solution; accept a worse move with probability $e^{-\Delta/T}$, cooling $T \leftarrow \alpha T$	combinatorial (VLSI placement, scheduling)
Particle swarm	bird flocking	velocity drawn toward personal and global best	continuous parameter tuning
Ant colony	ant foraging	pheromone τ on edges; $p_{ij} \propto \tau^\alpha \eta^\beta$, then evaporate and reinforce	routing, TSP
Artificial bee colony	honey-bee foraging	employed, onlooker and scout bees	continuous, numerical
Differential evolution	—	mutation by scaled vector differences	continuous; a robust default
Grey wolf, firefly, cuckoo	pack hunting, bioluminescence, brood parasitism	various attraction/hierarchy rules	engineering design, multi-modal problems

Simulated annealing's mechanism — at high T almost any uphill move is accepted (exploration); as T falls, acceptance collapses and the method becomes pure hill-climbing (exploitation). The cooling schedule performs the handover automatically.

PSO's overshoot — $v \leftarrow wv + c_1 r_1(p - x) + c_2 r_2(g - x)$ typically carries the particle *past* the global best — deliberately, so the swarm explores beyond the incumbent rather than merely converging onto it.

ACO's balance — evaporation (ρ) is the exploration term, preventing lock-in to early tours; reinforcement (Q/L , larger for *shorter* tours) is the exploitation term.

Making the Exact-versus-Heuristic Decision

1. **Problem in P with a known algorithm** (sorting, shortest path, MST, matching, max flow) → use that algorithm. Never use a GA to sort.
2. **NP-hard but small** ($n \lesssim 30$) **or heavily constrained** → backtracking, branch and bound, or a SAT/ILP solver — these still return the exact optimum.
3. **NP-hard, large, objective cheap to evaluate** → a metaheuristic matched to the problem type.
4. **Always report the gap to a bound** (an LP relaxation, or the MST lower bound for TSP), so that a “good enough” claim is quantified rather than asserted.

Other responses to NP-hardness — approximation algorithms with a proven ratio (2-approximation for vertex cover, 1.5 for metric TSP); restriction to tractable special cases (trees, bipartite graphs, bounded treewidth); and exact exponential algorithms with strong pruning.

💡 Exam Tip & Key takeaway: Block D — The Thread That Connects Both Units

- **Recurrences are the bridge.** Unit V’s master theorem exists largely to analyse Unit VI’s divide-and-conquer designs.
- **NP-completeness is the motivation.** Metaheuristics are not a first resort; they are the response to problems Unit V proves intractable.
- **Optimal substructure appears twice** — as the property justifying greedy and dynamic programming, and as the reason recurrences decompose at all.
- **The pseudo-polynomial trap** ($\Theta(nW)$ for knapsack) is where the two units collide most sharply: a table algorithm that looks polynomial but is not, because complexity is measured in the *encoded* input length.
- **Backtracking is DFS** from Unit IV, plus pruning; DPLL is backtracking plus resolution from Unit I. Almost nothing in Unit VI is genuinely new machinery — it is the earlier units recombined.